35. Edge Case (Uç Durum) Avcılığı: AI ile Olmayacak Senaryoları Bulmak

29 Şubat, Artık Saniyeler, Boş Stringler ve Sonsuz Sayılarla Sistemi Kırmak
Bölüm 6 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Dünyanın en iyi yazılımcılarının yazdığı kodlar bile neden 29 Şubat'ta veya sıfıra bölme (division by zero) durumlarında çöker ve AI bu kör noktaları nasıl yakalar?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Kullanıcılar asla sizin düşündüğünüz gibi davranmaz; beklenmedik girdiler sistemin gerçek stres testidir. - 📌 Edge Case (Uç Durum): Boş stringler, maksimum integer limitleri, farklı zaman dilimleri (Timezones) ve özel karakterlerdir. - 📌 AI modellerine 'Bu sistemi patlatacak 10 anormal sınır durum üret' demek, en ucuz ve etkili güvenlik kalkanıdır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir bankomat (ATM) cihazı düşünün.

Mühendisler ATM'yi test ediyor: - Kartı sok, şifreyi gir, 100 TL çek -> Başarılı! - 500 TL çek -> Başarılı!

Peki sarhoş bir müşteri gelip ATM'ye: - Eksi 100 TL (-100 TL) çekmek isterse ne olur? (Yazılım eksi parayı çıkaramayacağı için kullanıcının hesabına para ekleyebilir!) - Kart yuvasına bozuk para veya kağıt mendil tıkarsa? - Aynı anda hem interneti kesip hem para çekme tuşuna basarsa?

İşte buna **Edge Case (Uç Durum)** denir. Acemi geliştirici sadece güneşli havaları düşünür. Kıdemli mühendis ise fırtınada çatının uçup uçmayacağını test eder.

🧠 2. AI Çağında Yeni Zihniyet Modeli

En Meşhur 5 Edge Case Kategorisi:

```
1. SAYISAL SINIRLAR (Numeric Limits):
   - Sıfır (0), Negatif Sayılar (-1), Ondalık Yuvarlama (0.1 + 0.2),
MAX_SAFE_INTEGER.

2. METİN VE KARAKTERLER (Strings & Unicode):
   - Boş string (""), Sadece boşluk ("   "), Null/Undefined, Emojiler (👨‍👩‍👧‍👦), SQL/HTML
etiketleri.

3. ZAMAN VE TARİH (Temporal & Clocks):
   - 29 Şubat (Artık yıl), Gün ışığından yararlanma (Saatlerin 1 saat geri/ileri
alınması), UTC vs Yerel Saat.

4. KOLEKSİYONLAR (Collections & Arrays):
   - Boş liste ([]), Tek elemanlı liste ([1]), 1.000.000 elemanlı liste.

5. EŞZAMANLILIK (Concurrency & Race Conditions):
   - Aynı butona 10 milisaniye içinde 2 kez çift tıklama!
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Kaos Mühendisi ve Uç Durum (Edge Case) Avcısı
Aşağıdaki kullanıcı yaşını ve doğum gününü doğrulayan fonksiyonu incele.

## KOD:
```typescript
function isEligibleToVote(birthDateStr: string): boolean {
  const birthYear = new Date(birthDateStr).getFullYear();
  const currentYear = new Date().getFullYear();
  return (currentYear - birthYear) >= 18;
}
````

GÖREV:

1. Bu koddaki en az 4 ölümcül sınır durumu (edge case) tespit et (Örn: Ay ve gün hesabı, 29 Şubat, geçersiz tarih formatı, gelecekteki tarihler).
2. Bu sınır durumları patlatacak Jest test senaryolarını yaz.
3. Fonksiyonu tüm bu uç durumlara karşı kurşun geçirmez şekilde refactor et.
````

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Kötü Kod: Sadece yılı çıkarır; ay ve günü hesaba katmaz!
// Örnek: Bugün Ocak 2026 ise ve kullanıcı Aralık 2008 doğumluysa (henüz 17 yaşındayken) 18 kabul eder!
export function isEligibleToVoteNaive(birthDateStr: string): boolean {
  return (new Date().getFullYear() - new Date(birthDateStr).getFullYear()) >= 18;
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```typescript
export function isEligibleToVoteRobust(
  birthDateStr: string,
  now: Date = new Date()
): boolean {
  const birthDate = new Date(birthDateStr);

  // 1. Uç Durum: Geçersiz tarih formatı (NaN)
  if (isNaN(birthDate.getTime())) {
    throw new TypeError("Geçersiz tarih formatı girildi.");
  }

  // 2. Uç Durum: Gelecekte bir doğum tarihi girilemez
  if (birthDate > now) {
    throw new RangeError("Doğum tarihi bugünden ileri bir tarih olamaz.");
  }

  // 3. Kesin Yaş Hesabı (Ay ve Gün Hassasiyeti)
  let age = now.getFullYear() - birthDate.getFullYear();
  const monthDiff = now.getMonth() - birthDate.getMonth();

  if (monthDiff < 0 || (monthDiff === 0 && now.getDate() < birthDate.getDate())) {
    age--; // Doğum günü bu yıl henüz gelmedi!
  }

  // 4. Uç Durum: 130 yaşından büyük mantıksız değerler
  if (age > 130) {
    throw new RangeError("Doğrulanamaz yaş değeri.");
  }

  return age >= 18;
}
```

💡 **Kod Analizi:** Kıdemli sürüm; geçersiz tarihleri (`NaN`), henüz günü gelmemiş ayları, gelecekteki doğum tarihlerini ve mantıksız sınırları tek tek kontrol ederek çökmeleri engeller.
```

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Kullanıcının her zaman doğru formatta veri gireceğini varsaymak.
  - **Çözüm:** Girdileri daima şüpheyle karşılayın; 'Boş gelirse ne olur?', 'Çok büyük gelirse ne olur?' diye sorun.
- **Tuzak:** Tarih hesaplamalarında `getFullYear() - getFullYear()` yapıp gün farkını unutmak.
  - **Çözüm:** Tarih karşılaştırmalarında gün ve saat hassasiyetini veya güvenilir kütüphaneleri (date-fns) kullanın.
- **Tuzak:** Sayısal işlemlerde sıfıra bölme (division by zero) kontrolü yapmamak.
  - **Çözüm:** Her bölme işleminden önce paydanın sıfır olmadığını teyit edin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 35: Edge Case Avcılığı - AI ile Olmayacak Senaryoları Bulmak*

- 🖥️ **Slayt 1:** Neden en büyük sistem çöküşleri Edge Case'lerden kaynaklanır?
- 🖥️ **Slayt 2:** 5 Temel Sınır Durum Kategorisi (Sayılar, Metinler, Tarihler, Listeler, Concurrency).
- 🖥️ **Slayt 3:** AI'ı 'Kötü Niyetli Hacker' rolünde sınır testleri için kullanmak.
- 🖥️ **Slayt 4:** Canlı Demo: Hatalı yaş hesaplama fonksiyonunu kırmak ve onarmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Basit bir metin kısaltma (truncate) fonksiyonunu AI'a verin. 'Bu fonksiyonu çökertecek 5 sınır durum testi yaz' deyin. AI'ın boş string, 100.000 karakterlik metin ve çoklu bayt emoji testlerini nasıl ürettiğini ve kodun patladığını canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki en kritik bir fonksiyonu seçin. AI'a 'Bu fonksiyonu production'da çökertecek 5 çılgın edge case üret' deyin ve çıkan testleri çalıştırın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: JavaScript/TypeScript'te `0.1 + 0.2 === 0.3` ifadesinin sonucu neden `false` döner? - A) JavaScript bozuk bir dildir - B) Sayılar ikili (binary floating-point) tabanda temsil edildiği için 0.3 yerine 0.30000000000000004 hassasiyet kaybı oluşur - C) Sadece Google Chrome'da öyle olur - D) Sonuç true döner > **Doğru Yanıt:** B > **Açıklama:** IEEE 754 kayan noktalı standartta ikili taban kesirli sayıları tam temsil edemez; bu nedenle parasal işlemlerde kuruş yuvarlama kontrolleri şarttır.

Soru 2: Bir fonksiyonda 'Sıfıra Bölme' (Division by Zero) kontrolü yapılmadığında çoğu dilde ne gerçekleşir? - A) Kod daha hızlı çalışır - B) `Infinity` döner veya `ZeroDivisionError` fırlatılarak program anında çöker - C) Bilgisayarın ekranı kararır - D) Sayı otomatik olarak 1 olur > **Doğru Yanıt:** B > **Açıklama:** Sıfıra bölme matematiksel olarak tanımsızdır; diller ya çalışma zamanı hatası fırlatır ya da beklenmedik `Infinity` / `NaN` üreterek sistemi bozar.